

Simulation Execution Standard - (SES)

OOF™ Origin Open Foundation™

Independent Methodological Authority

Simulation Execution Standard (SES) defines the governance conditions under which an approved Simulation Object, Purpose & Scope, Reality Representation, Assumption Basis, Model & Environment, Input & Data Basis, and Scenario Set are instantiated into an actual simulation run or campaign, operationally controlled during execution, observed without losing material execution context, repeated where required, and documented as a traceable Simulation Execution Record.

SES governs the transition from:

Simulation Design

to:

Simulation Activity.

Its purpose is to ensure that a simulation does not become an undocumented computational event whose relationship to its intended configuration, inputs, scenarios, execution conditions, observations, repetitions, deviations, and resulting outputs can no longer be reconstructed.

SES does not determine whether the simulation result is valid, correct, reliable, or operationally applicable.

It governs:

what was actually executed, under which governed conditions, what occurred during execution, whether repetition was controlled, and whether the execution can later be reconstructed sufficiently for downstream governance.

Architectural Position

SES is the eighth Parent Standard of SIMULOS®.

It occupies the transition point between:

Simulation Scenario Governance

and:

Simulation Output Governance.

The architecture progresses:

Simulation Object

↓

Simulation Purpose & Scope

↓

Reality Representation

↓

Simulation Assumptions

↓

Simulation Model & Environment

↓

Simulation Inputs & Data

↓

Simulation Scenarios

↓

Simulation Execution

↓

Simulation Output Governance

↓

Downstream Simulation Lifecycle Governance

SES therefore represents the point at which the governed simulation specification becomes an executed simulation event.

Governance Flow Position

Upstream Input:

Governed Scenario Space + Governed Simulation Configuration + Governed Input Basis

↓

Simulation Instantiation



Execution Control



Simulation Observation



Simulation Repetition



Execution Documentation



Governed Simulation Execution Record



Downstream Output Governance

Operational Space

SES governs: simulation-run instantiation, simulation-campaign instantiation, execution identity, execution configuration, object-version binding, model-version binding, environment-version binding, scenario binding, input binding, parameter binding, assumption binding where material, execution authorization, execution initiation, execution state, execution sequencing, execution controls, runtime conditions, execution interventions, execution deviations, execution interruption, execution restart, execution termination, execution completion, simulation observation, observation context, observation timing, event capture, anomaly capture, state-transition capture, repeated execution, repetition conditions, randomization control, seed governance where applicable, repeatability conditions, reproducibility conditions, execution comparison, execution documentation, execution provenance, execution traceability, execution completeness, execution lineage, and formation of the Governed Simulation Execution Record.

Operational Scope

SES applies to simulation execution across environments including: AI systems, autonomous systems, robotics, multi-agent systems, aerospace and space systems, industrial systems, digital twins, infrastructure systems, transportation systems, scientific and engineering simulations, financial and economic simulations, safety simulations, mission simulations, training simulations, synthetic environments, hardware-in-the-loop environments, software-in-the-loop environments, and future simulation systems.

SES is technology-neutral.

It does not prescribe a particular: simulator, simulation engine, programming language, computational infrastructure, numerical method, orchestration platform, or execution technology.

Execution Object

Every governed simulation execution should be distinguishable as an identifiable execution object.

Depending upon implementation, this may be:

Simulation Run

Simulation Episode

Simulation Trial

Simulation Batch

Simulation Campaign

or another defined execution unit.

The governing requirement is not terminology.

The governing requirement is:

the executed activity must be sufficiently identifiable to reconstruct what actually occurred.

Simulation Execution ID

A materially relevant simulation execution should have a stable identifier capable of connecting the execution to its governing basis.

Conceptually:

Simulation Execution ID

↓

Simulation Object Version

Purpose & Scope

Reality Representation

Assumption Basis

Model Version

Environment Version

Input Set

Scenario ID

Parameter Configuration

Execution Conditions

Execution Record

Produced Outputs

This prevents simulation results from becoming detached from the execution that produced them.

Instantiation Principle

A simulation specification does not become an executed simulation merely because its components exist.

SES distinguishes:

Defined Simulation Configuration

from:

Instantiated Simulation Configuration.

Instantiation establishes which exact versions, parameters, inputs, scenarios, environments, and relevant conditions were assembled for a specific execution.

Configuration Binding

Before execution, materially relevant components should be bound to the execution instance.

This may include: Simulation Object version, model version, environment version, scenario version, input dataset, parameter set, software configuration, hardware configuration, random seed, dependency state, external service configuration, and execution timestamp.

No Assumed Configuration Principle

SIMULOS® establishes:

The configuration intended to run must not automatically be treated as the configuration that actually ran.

Where material, execution records should establish the instantiated configuration rather than relying solely upon pre-execution intent.

Execution Authorization

Where organizational, technical, safety, financial, or regulatory consequence requires authorization, SES may require an identifiable authorization basis before execution.

Authorization may govern: permitted scenario classes, resource limits, execution environment, access permissions, external-system interaction, sensitive data, hardware use, operational interfaces, and termination authority.

Execution Initiation

Execution Initiation marks the transition from prepared simulation configuration to active simulation.

A governed initiation record may establish: execution identity, initiation time, configuration status, scenario, inputs, environment, authorization, and initial state.

Execution State

SES may distinguish execution states such as:

Prepared

Initialized

Running

Paused

Interrupted

Restarted

Completed

Terminated

Failed

or:

Execution State Unknown.

These states allow downstream users to distinguish a successfully completed simulation from a partial or interrupted execution.

Execution Control

Simulation Execution Control governs the mechanisms through which execution remains within the conditions established for the simulation.

Controls may concern: execution sequence, timing, resource use, synchronization, scenario progression, parameter application, dependency behavior, state transitions, interventions, runtime limits, termination criteria, and recovery behavior.

Execution Control ≠ Outcome Control

A critical SIMULOS® distinction is:

Execution Control governs how the simulation is performed. It must not become manipulation designed to force a preferred simulation outcome.

Unexpected or unfavorable behavior should remain observable rather than being silently corrected merely because it conflicts with expectations.

Runtime Intervention

A simulation may require intervention during execution.

Examples include: pause, parameter adjustment, injected failure, scenario escalation, external input, human intervention, recovery command, or controlled termination.

Where intervention can materially affect the result, it should remain traceable.

No Invisible Intervention Principle

SIMULOS® establishes:

A material intervention during simulation execution must not disappear from the execution history.

Execution Deviation

An Execution Deviation exists where actual execution materially differs from the intended execution basis.

Examples include: wrong configuration, missing dependency, delayed input, unexpected environment state, parameter mismatch, runtime error, unscheduled intervention, timing failure, resource exhaustion, incomplete scenario execution, or execution-system failure.

Deviation Does Not Automatically Invalidate Execution

SES does not predetermine the consequence of a deviation.

Instead it requires the deviation to remain visible.

A deviation may be:

Immaterial

Material but Acceptable

Conditionally Acceptable

Execution-Limiting

Execution-Invalidating

or:

Undetermined

depending upon downstream assessment.

Execution Observation

Simulation Observation governs the structured capture of materially relevant events and states during execution.

Observation may concern: system state, environment state, agent behavior, transitions, failures, recoveries, interactions, timing, resource consumption, control events, interventions, anomalies, and emergent behavior.

Observation ≠ Interpretation

SES establishes a strong boundary:

Observation records what occurred during execution. Interpretation determines what that occurrence means.

The latter belongs primarily to downstream output and analytical governance.

Observation Context

An observation without sufficient context may become misleading.

Where material, observations should preserve:

What happened

When it happened

Where in the simulation it happened

Under which configuration

Under which scenario

Under which inputs

Under which execution state

and:

What preceded or followed the event.

Observation Resolution

Not every simulation requires identical observation resolution.

Observation may be: continuous, periodic, event-triggered, threshold-triggered, state-triggered, sampled, or selectively captured.

The appropriate resolution depends upon the Simulation Purpose and materially relevant execution phenomena.

Observation Blind Spots

SES should preserve known observation limitations.

Examples include: unlogged internal states, insufficient sampling frequency, unavailable telemetry, inaccessible agent reasoning, missing environment events, or data lost during execution.

A simulation should not later be represented as fully observed where material execution regions were not observable.

Execution Repetition

A simulation may be repeated to examine: consistency, variability, stochastic behavior, uncertainty, scenario sensitivity, parameter sensitivity, rare events, or distributional behavior.

SES governs how repetitions remain distinguishable and comparable.
Repeatability

Repeatability concerns whether materially equivalent execution conditions can produce sufficiently comparable execution behavior when repetition is intended to preserve those conditions. Reproducibility

Where applicable, Reproducibility concerns whether another execution environment, actor, system, or implementation can recreate the materially relevant simulation execution basis sufficiently to examine comparable behavior.

SIMULOS® does not assume every simulation must be perfectly reproducible.

Some simulations contain: stochastic processes, nondeterministic hardware, external services, adaptive agents, distributed systems, real-time human interaction, or changing environments.

The requirement is to govern the conditions affecting reproducibility rather than falsely claim determinism.

Randomness Governance

Where randomness materially affects execution, SES may govern: randomization method, seed identity, seed generation, seed reuse, stochastic parameterization, sampling state, and relationship between randomization and repetitions.

Seed Traceability

Where technically applicable and materially relevant, preserving a random seed may allow an execution to be reconstructed.

However:

Seed preservation alone does not establish reproducibility

if other execution conditions have changed.

Repetition Identity

Every repeated execution should remain distinguishable.

For example:

Scenario S-104

may produce:

Run S-104-R001

Run S-104-R002

Run S-104-R003

and so forth.

The exact naming system is implementation-specific.

Controlled Repetition

SES distinguishes:

Controlled Repetition

from:

Uncontrolled Re-Running.

Controlled repetition preserves sufficient information to determine: what remained constant, what changed, why it changed, and whether resulting executions may legitimately be compared.

Repetition Variance

Repeated simulations may intentionally vary: seed, input, parameter, environment, agent population, timing, scenario condition, or another dimension.

Such variation should remain distinguishable from accidental execution drift.

Execution Drift

Execution Drift occurs where repeated simulations cease to represent materially comparable execution conditions without that change being sufficiently governed.

Possible causes include: software updates, infrastructure changes, model changes, environment updates, dependency changes, altered defaults, parameter changes, or orchestration changes.

Execution Comparability

Two simulation executions should not be treated as directly comparable merely because they share the same Scenario ID.

Comparability may also depend upon: model version, environment version, input basis, parameterization, randomization, execution controls, and material runtime conditions.

Simulation Campaign Governance

Large simulation programs may execute: thousands, millions, or billions

of simulation runs.

SES allows these runs to be governed as a Simulation Campaign while preserving sufficient run-level traceability where material.

Campaign Identity

A Simulation Campaign may bind:

Campaign ID

↓

Simulation Purpose

↓

Scenario Set

↓

Execution Configuration

↓

Run Population

↓

Repetition Logic

↓

Execution Controls

↓

Campaign Record Campaign-Level Control

Campaign-level governance may establish: permitted configurations, scenario population, execution infrastructure, resource limits, repetition rules, stopping conditions, logging requirements, exception handling, and documentation requirements.

Run-Level Exceptions

A campaign may be generally conformant while individual runs contain deviations.

SES should therefore preserve the distinction between:

Campaign Status

and:

Individual Execution Status.

Execution Completeness

Execution Completeness asks:

Was the simulation activity required by the instantiated execution basis actually completed sufficiently?

Possible states may include:

Complete

Conditionally Complete

Partially Complete

Incomplete

Failed

or:

Undetermined. Completion ≠ Successful Outcome

A simulation may execute completely and reveal catastrophic system failure.

That may still represent a fully completed simulation execution.

Likewise, a simulation producing an apparently desirable output may have been incompletely executed.

SIMULOS® therefore establishes:

Execution Completeness and Simulation Outcome are separate governable questions.

Execution Documentation

Simulation Execution Documentation preserves the execution history sufficiently for later reconstruction.

Documentation may include: Execution ID, Campaign ID, object version, model version, environment version, scenario, inputs, parameters, assumptions where material, authorization, start time, completion time, execution state, interventions, deviations, observations, repetition relationship, randomization basis, infrastructure state, completion status, and output references.

Execution Provenance

Execution Provenance establishes where a Simulation Execution Record came from and which technical and governance basis produced it.

Execution Lineage

Execution Lineage connects related runs.

For example:

Baseline Run

↓

Repeated Run

↓

Modified Parameter Run

↓

Stress Escalation Run

↓

Recovery Run

This allows simulation evolution to be reconstructed rather than viewed as disconnected output files.

Point-in-Time Reconstruction

SES should enable questions such as:

Which exact model version ran?

Which scenario version was instantiated?

Which inputs were used?

Which parameters were active?

Which environment version existed?

Was the execution interrupted?

Was there a runtime intervention?

Was the run repeated?

Which seed was used where applicable?

Were material anomalies observed?

Was the execution complete?

Which outputs originated from this execution?

No Orphan Output Principle

SIMULOS® establishes:

A material Simulation Output should remain traceable to the Simulation Execution that produced it.

An output detached from its execution basis loses critical interpretive context.

No Successful-Run Selection Bias

Where a Simulation Campaign produces both expected and unexpected executions, SES should preserve material failed, interrupted, anomalous, and unfavorable runs.

SIMULOS® establishes:

Simulation Execution governance must not preserve only the runs that support the preferred conclusion.

Failed Executions as Information

Failed executions may reveal: infrastructure weaknesses, scenario incompatibility, model instability, resource constraints, timing problems, or

previously unknown simulation limitations.

SES therefore treats execution failure as governable information rather than disposable noise.

Autonomous Simulation Execution

AI and autonomous systems may increasingly: generate simulations, select scenarios, instantiate environments, modify parameters, execute campaigns, identify anomalies, and launch follow-up simulations automatically.

SES remains applicable.

The execution record should preserve where material:

Human-Initiated Execution

AI-Initiated Execution

Autonomously Triggered Execution

or another relevant execution authority state.

Adaptive Simulation

An adaptive simulation may alter future execution based upon earlier results.

For example:

Run

↓

Observation

↓

Adaptive Scenario Selection

↓

New Run

↓

New Observation

SES should preserve this causal execution chain.

No Hidden Adaptive Path Principle

Where prior simulation results materially determine subsequent execution selection, the adaptive path should remain reconstructable.

Otherwise the resulting campaign may appear predetermined when it was actually dynamically generated.

Human-in-the-Loop Simulation

Where humans participate during simulation, SES may preserve: participant role, intervention authority, intervention timing, decisions, response delay, override, and materially relevant human actions.

Hardware-in-the-Loop Simulation

For hardware-in-the-loop environments, execution records may additionally preserve: hardware identity, hardware configuration, firmware version, calibration state, communication state, timing characteristics, and physical interface conditions.

Software-in-the-Loop Simulation

Software-in-the-loop execution may preserve: software build, dependencies, runtime environment, API versions, model versions, configuration, and orchestration state.

Space and Mission Simulation

Space and aerospace simulation programs may execute enormous campaigns across: trajectories, navigation uncertainty, environmental conditions, communication states, propulsion states, sensor states, subsystem failures, autonomy states, and recovery conditions.

SES governs not merely whether these simulations were launched but whether the execution history can establish:

which mission configuration actually ran under which conditions and what occurred during that run.

This becomes especially important when simulation campaigns are used to support decisions that cannot later be easily tested in operational reality.

Autonomous Systems

For autonomous systems, execution governance becomes particularly important where behavior can depend upon: internal state, prior actions, memory, stochasticity, environment changes, external agents, timing, permissions, and adaptive decision paths.

Two runs using nominally identical scenarios may diverge materially.

SES preserves the conditions necessary to understand that divergence.

Execution Bottleneck

In mature simulation programs, the bottleneck may not be computational capacity.

Organizations may already possess enormous simulation infrastructure.

The bottleneck may instead be:

execution traceability across millions of runs.

Typical questions become:

Which configuration produced this result?

Can we reproduce it?

Was this run interrupted?

Was there an intervention?

Which environment version was active?

Which outputs belong to which execution?

Did failed runs disappear from analysis?

Did execution conditions drift across the campaign?

SES turns these questions into an explicit governance space.

Relationship to Simulation Scenario Standard

SSS determines:

what should be simulated and whether materially relevant Scenario Space is sufficiently covered.

SES determines:

how those scenarios are actually instantiated and executed.

Therefore:

Scenario Coverage $\neq$ Execution Completion.

A scenario may exist in the governed Scenario Set without ever being successfully executed.

Relationship to Simulation Model & Environment Standard

SMES governs construction, configuration, boundaries, parameterization, and versioning of the simulation model and environment.

SES binds a specific governed model and environment configuration to an actual execution.

Relationship to Simulation Input & Data Standard

SIDS governs which data and inputs may enter the simulation and under which conditions.

SES records which governed inputs actually entered a specific execution.

Relationship to Simulation Assumption Standard

SAS governs the role of assumptions inside the simulation.

SES preserves the applicable assumption basis where it materially affects execution reconstruction.

Relationship to Simulation Output Governance Standard

SES ends by producing the governed execution basis from which simulation outputs emerge.

SOGS begins by asking:

What exactly did the simulation produce, how should those outputs be classified and bounded, and what may legitimately be inferred from them?

The handoff is therefore:

Governed Simulation Execution Record

↓

Simulation Output

↓

Simulation Output Governance Relationship to VALIDOS®

VALIDOS® may later determine whether simulation execution records and outputs are sufficient for a particular Validation Purpose.

SES does not make that Validation Determination.

This preserves the boundary:

Simulation Execution Governance ≠ Validation Governance.

Relationship to INTEGROS®

INTEGROS® may govern integrity properties of execution records, artifacts, or related information.

SES does not duplicate general integrity governance.

SES governs the simulation-specific execution lifecycle.

Relationship to ADIT®

ADIT® may audit evidence and records produced by simulation execution.

SIMULOS® owns:

simulation execution itself.

ADIT® may subsequently ask whether the execution record is sufficiently auditable, evidenced, or reconstructable for an audit purpose.

Relationship to ORA™

ORA™ governs actual Operational Reality.

SES governs activity occurring inside the simulation environment.

An execution that perfectly conforms to its simulation environment does not establish that the environment accurately represents operational reality.

Relationship to ASGA™

ASGA™ may define governance requirements concerning autonomous authority, permissions, override, escalation, coordination, and continuity.

SES may instantiate and observe scenarios exercising those conditions without taking ownership of autonomous-system governance itself.

Execution Governance Boundary

SES begins when the upstream simulation basis is sufficiently established for a scenario or simulation campaign to be instantiated.

SES ends when the required simulation activity has been: instantiated, executed, controlled, observed, repeated where required, documented, linked to material deviations, assigned an execution-completeness status, and preserved as a Governed Simulation Execution Record capable of supporting downstream Simulation Output Governance.

SES does not determine: whether the simulation model is universally valid, whether Reality Representation is sufficient, whether the resulting output is correct, whether an inference from the output is legitimate, whether operational deployment should occur, whether a Validation Object is valid, or whether a system is safe.

Parent Standard Modules 1. SIM — Simulation Instantiation Module

Governs how a defined simulation basis becomes a specific executable simulation instance.

It establishes the binding between:

Execution ID → Simulation Object → Model → Environment → Scenario → Inputs → Parameters → Applicable Conditions

Its primary question is:

What exactly is being instantiated for execution? 2. SECM — Simulation Execution Control Module

Governs how active simulation execution is initiated, controlled, progressed, interrupted, modified where permitted, terminated, and completed.

Its primary question is:

How must the simulation remain controlled while it is actually running? 3. SOM — Simulation Observation Module

Governs how materially relevant execution states, events, behaviors, transitions, anomalies, interventions, and runtime conditions are observed and preserved.

Its primary question is:

What materially relevant events occurred during execution, and under which execution context? 4. SRPM — Simulation Repetition Module

Governs repeated execution, randomization, repeatability, reproducibility conditions, comparison, execution variance, and controlled repetition.

Its primary question is:

When a simulation is repeated, what must remain constant, what may change, and how can repeated executions remain meaningfully comparable? 5. SEDM — Simulation Execution Documentation Module

Governs the documentation, provenance, lineage, completeness, traceability, and reconstructability of simulation execution.

Its primary question is:

Can the executed simulation later be reconstructed sufficiently to determine what actually ran and what occurred?

Parent Standard Internal Flow

SIM — Simulation Instantiation

↓

SECM — Simulation Execution Control

↓

SOM — Simulation Observation

↓

SRPM — Simulation Repetition

↓

SEDM — Simulation Execution Documentation

↓

Governed Simulation Execution Record

This sequence transforms:

simulation intent

into:

traceable simulation execution. Minimum Implementation Framework 1. Identify the Execution

Assign a Simulation Execution ID or equivalent stable execution identity. 2. Bind the Execution Configuration

Record the materially relevant: Simulation Object, model, environment, scenario, inputs, parameters, versions, and conditions.

3. Establish Execution Authorization

Where required, establish who or what may initiate and control execution. 4. Instantiate the Simulation

Create the actual executable simulation instance. 5. Execute Under Defined Controls

Apply relevant runtime, sequencing, intervention, resource, and termination controls. 6. Observe Material Execution Events

Capture materially relevant states, transitions, anomalies, interventions, failures, and other execution events. 7. Govern Deviations

Identify differences between intended and actual execution. 8. Govern Repetition

Where execution is repeated, preserve what remained constant and what materially changed. 9. Assess Execution Completeness

Determine whether execution was: Complete, Conditionally Complete, Partially Complete, Incomplete, Failed, or Undetermined.

10. Preserve Execution Documentation

Create sufficient documentation to reconstruct the execution basis and history. 11. Link Execution to Outputs

Ensure resulting Simulation Outputs remain connected to the executions that produced them. 12. Establish the Governed Simulation Execution Record

Produce the controlled execution basis for downstream Simulation Output

Governance Outputs

SES may produce: Simulation Execution ID, Simulation Campaign ID, Simulation Instantiation Record, Execution Configuration Record, Object-Version Binding, Model-Version Binding, Environment-Version Binding, Scenario Binding Record, Input Binding Record, Parameter Binding Record, Execution Authorization Record, Execution Initiation Record, Execution State Record, Execution Control Record, Runtime Condition Record, Execution Intervention Record, Execution Deviation Record, Execution Interruption Record, Execution Restart Record, Execution Termination Record, Execution Completion Record, Simulation Observation Record, Event Observation Record, Anomaly Observation Record, State Transition Record, Observation Limitation Record, Simulation Repetition Record, Randomization Record, Seed Record, Repeatability Record, Reproducibility Condition Record, Repetition Variance Record, Execution Drift Record, Execution Comparability Record, Simulation Campaign Record, Campaign Exception Record, Execution Completeness Assessment, Execution Provenance Record, Execution Lineage Record, Execution Documentation Record, Execution-to-Output Traceability Record, and Governed Simulation Execution Record.

Enterprise Application

SES can be applied over an organization's existing simulation infrastructure without replacing the underlying simulator.

The organization may retain its: simulation engine, digital twin, HPC infrastructure, Monte Carlo platform, robotics simulator, mission simulator, AI evaluation environment, or proprietary simulation stack.

SES adds the governance layer that asks:

What actually ran?

Under which configuration?

Who or what initiated it?

What changed during execution?

What was observed?

Was it completed?

Was it repeated under comparable conditions?

Can the execution be reconstructed?

Can every material output be traced back to the execution that produced it?

This makes SES applicable as a simulation-execution governance and diagnostic framework without requiring organizations to abandon existing technical infrastructure.

Foundational Principle

A simulation result cannot be fully understood from its output alone; it must remain connected to the exact simulation configuration, execution conditions, observations, deviations, repetitions, and execution history that produced it.

Canonical Closing Statement

Simulation Execution Standard (SES) governs the point at which the designed simulation becomes an actual simulation event. Through governed instantiation, configuration binding, execution control, runtime observation, deviation preservation, repetition governance, randomization traceability, execution-completeness assessment, campaign governance, provenance, lineage, and execution documentation, SES establishes the conditions under which simulation activity remains reconstructable across individual runs and large-scale simulation campaigns. It prevents intended configuration from being confused with executed configuration, successful outcomes from being confused with complete execution, repeated runs from being assumed comparable without evidence, and Simulation Outputs from becoming detached from the executions that produced them. SES therefore transforms Simulation Execution from a transient computational event into a governed, traceable, and reusable operational record capable of supporting downstream Simulation Output Governance, validation, audit, and future simulation analysis.

Module Architecture

→ [Simulation Instantiation Module \(SIM\)](#)

→ [Simulation Execution Control Module \(SECM\)](#)

→ [Simulation Observation Module \(SOM\)](#)

→ [Simulation Repetition Module \(SRPM\)](#)

→ [Simulation Execution Documentation Module \(SEDM\)](#)

Related Documents

→ [About Simulation Execution Standard \(SES\)](#)

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the **MIP® Protection & Licensing** terms or **contact OOF® directly**.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/>